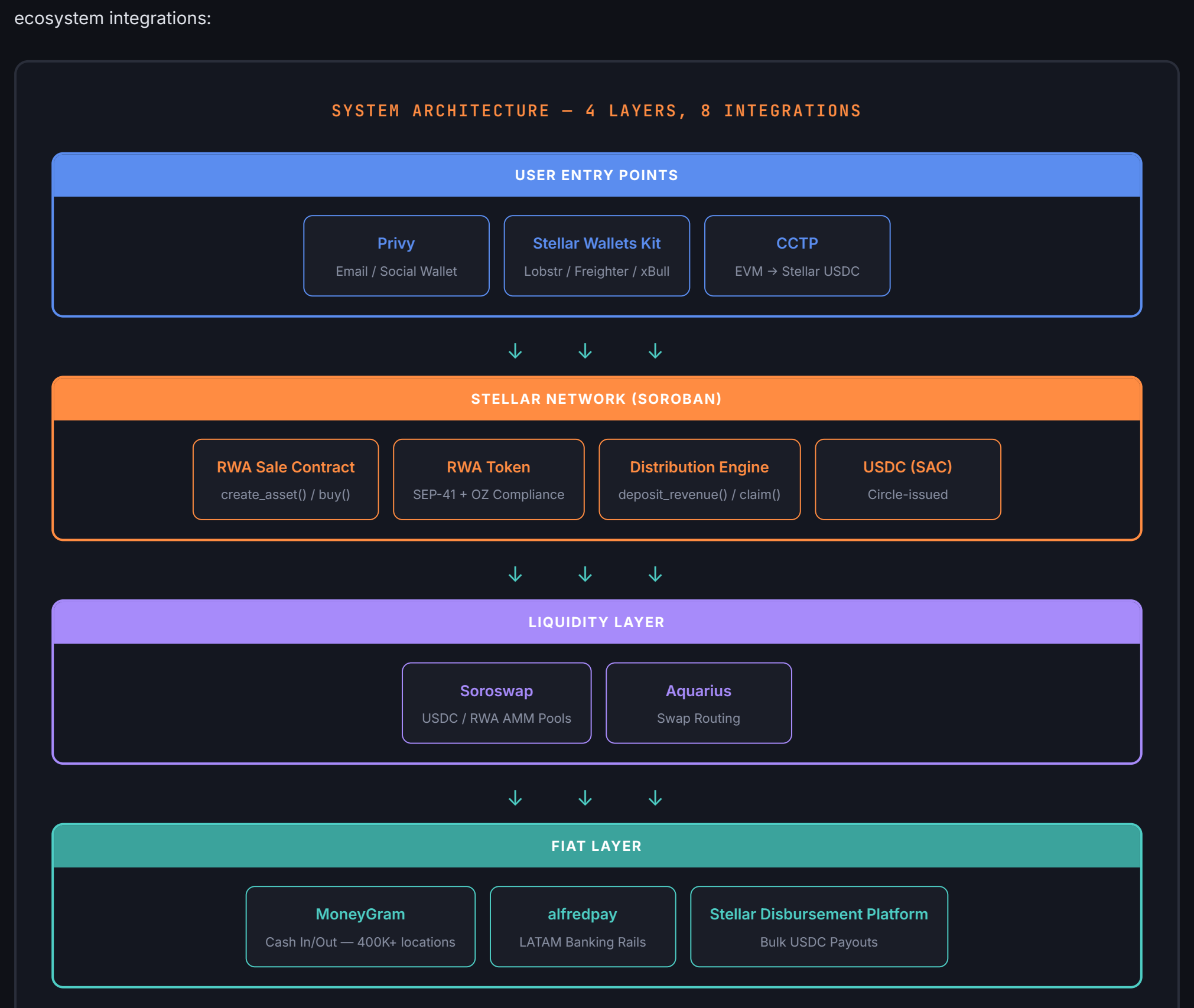
Tokeshare — Technical Architecture

LATAM Real-World Investing on Stellar

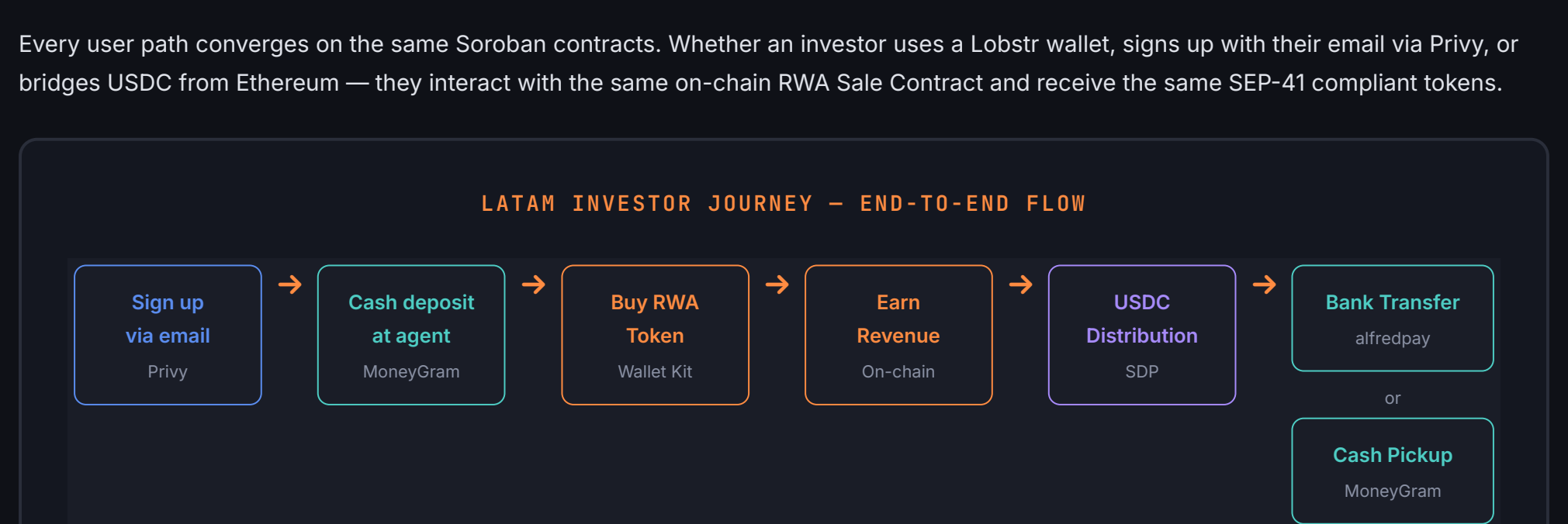
8 Stellar Ecosystem Integrations | Soroban Smart Contracts | Fiat On/Off-Ramp

1. System Overview

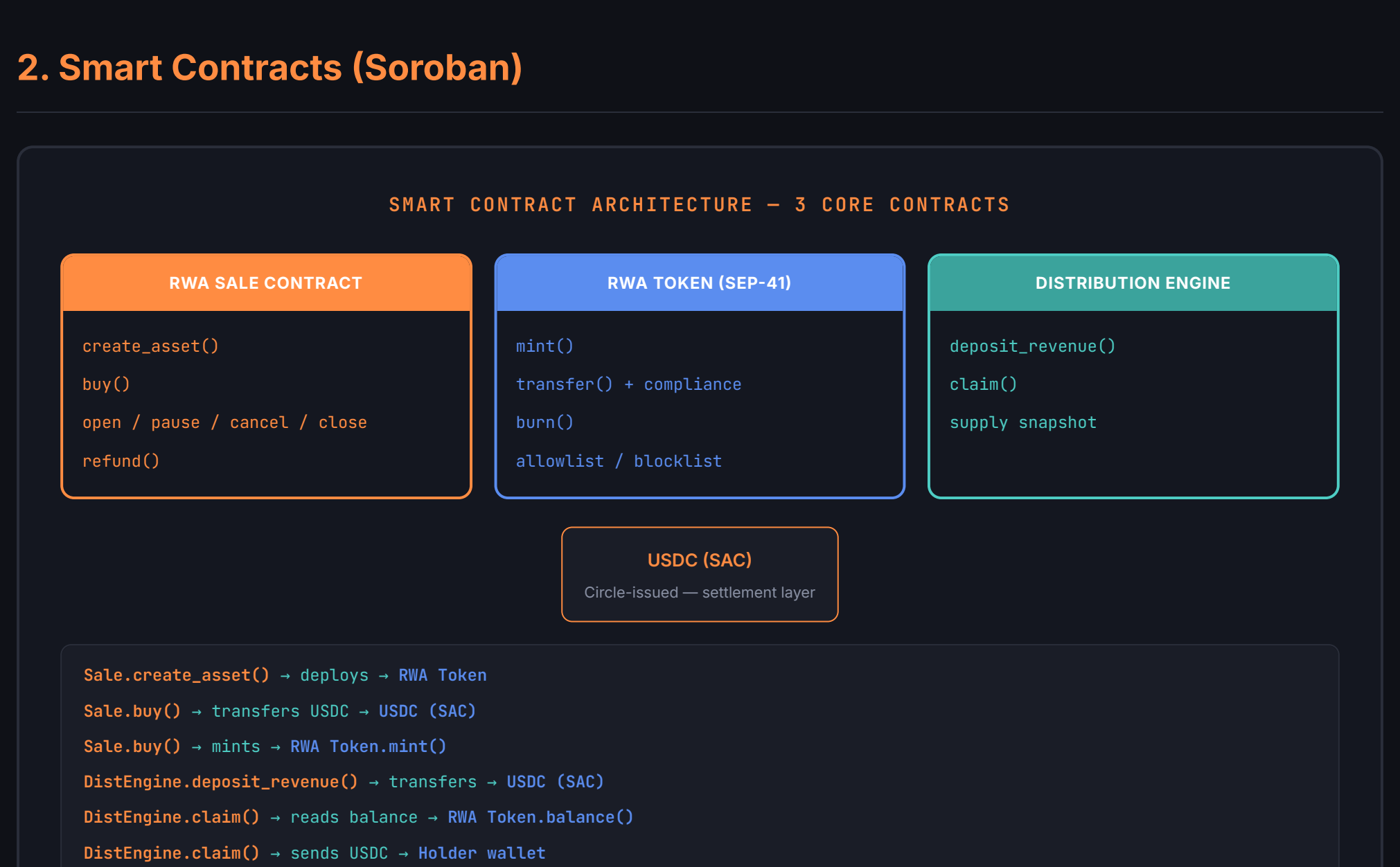
Tokeshare is a real-world asset (RWA) tokenization platform built on Stellar, enabling LATAM investors to buy fractional shares of real estate, restaurants, and other revenue-generating assets with as little as $10 USDC. The architecture connects four distinct layers through 8 Stellar ecosystem integrations:



Every user path converges on the same Soroban contracts. Whether an investor uses a Lobstr wallet, signs up with their email via Privy, or bridges USDC from Ethereum — they interact with the same on-chain RWA Sale Contract and receive the same SEP-41 compliant tokens.



2. Smart Contracts (Soroban)



```
Sale.create_asset() → deploys → RWA Token
Sale.buy() → transfers USDC → USDC (SAC)
Sale.buy() → mints → RWA Token.mint()
DistEngine.deposit_revenue() → transfers → USDC (SAC)
DistEngine.claim() → reads balance → RWA Token.balance()
DistEngine.claim() → sends USDC → Holder wallet
DistEngine.snapshot → reads → RWA Token.total_supply()
```

2.1 RWA Sale Contract — Extension of Existing Code

The sale contract is **already live** at `soroban/contracts/sale/src/lib.rs` (187 lines, 7 unit tests passing) and deployed on Stellar testnet at tokeshare.co/poc-stellar. The new version extends the existing contract with multi-asset management, lifecycle controls, and compliance hooks.

EXISTING (PRESERVED)	NEW (ADDED)
`buy()` — USDC → treasury + token → buyer	`create_asset()` — deploys RWA Token per asset
`quote()` — price calculation	`open_sale()` / `pause_sale()` / `close_sale()`
`available()` — remaining supply	`cancel_asset()` — cancellation + auto-refund trigger
`set_price()` — admin price update	`refund()` — automated USDC refunds on cancellation
Constructor (admin, token SAC, payment SAC, treasury)	Asset metadata: type, name, location, valuation, legal URI
Ceil division, Instance TTL management	Multi-asset management per `asset_id`
—	Event emission for all state changes

create_asset() Invocation

RUST / SOROBAN

```
create_asset(
    asset_type: "real_estate",      // or "restaurant", "vehicle", "commodity"
    name: "Angel Coeur Caribe",
    total_supply: 23_000,
    price_per_token: 10_0000000,    // 10 USDC (7 decimals)
    metadata_uri: "ipfs://Qm...",
    compliance_config: {
        require_allowlist: true,
        max_holders: 500,
        transfer_lockup_ledgers: 0
    },
    treasury: G...TREASURY,
    pay_sac: USDC_SAC_ADDRESS
)
→ deploys RWA Token contract
→ emits AssetCreated event
→ returns (asset_id, token_contract_address)
```

2.2 RWA Token — SEP-41 + OpenZeppelin Stellar Compliance

Each tokenized asset gets its own SEP-41 compliant token contract, deployed by the Sale Contract via `create_asset()`. The token implements the full SEP-41 interface (9 standard functions) extended with OpenZeppelin Stellar compliance modules for regulatory enforcement.

Compliance Features from OpenZeppelin Stellar

FEATURE	OZ MODULE	PURPOSE
Allowlist transfers	Allowlist	Only KYC-verified addresses can send/receive tokens
Blocklist enforcement	Blocklist	Sanctioned or flagged addresses permanently blocked
Capped supply	Capped	Total supply = asset valuation / token price, immutable after creation
Pausable	Pausable	Emergency halt of all transfers by admin

Transfer with Compliance Checks

RUST / SOROBAN

```
fn transfer(e: Env, from: Address, to: Address, amount: i128) {
    from.require_auth();
    check_nonnegative_amount(amount);

    // Compliance gates — all must pass
    require_not_paused(&e);
    require_allowed(&e, &from);
    require_allowed(&e, &to);
    require_not_blocked(&e, &from);
    require_not_blocked(&e, &to);

    // Execute transfer
    spend_balance(&e, from.clone(), amount);
    receive_balance(&e, to.clone(), amount);
    TokenEvents::transfer(&e, from, to, amount);
}
```

Mint with Supply Cap Enforcement

RUST / SOROBAN

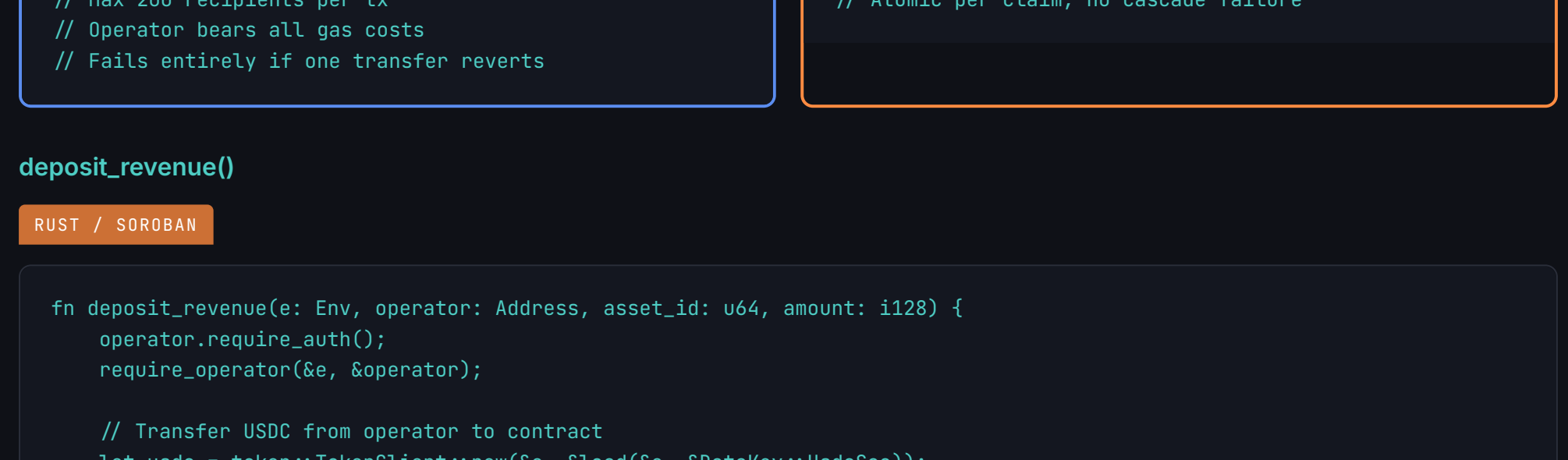
```
fn mint(e: Env, to: Address, amount: i128) {
    let admin = read_administrator(&e);
    admin.require_auth();
    check_nonnegative_amount(amount);

    // Enforce supply cap — cannot exceed asset valuation
    let supply = read_total_supply(&e);
    let cap = read_supply_cap(&e);
    if supply.checked_add(amount).expect("overflow") > cap {
        panic_with_error!(&e, Error::SupplyCapExceeded);
    }

    receive_balance(&e, to.clone(), amount);
    write_total_supply(&e, supply + amount);
    TokenEvents::mint(&e, to, amount);
}
```

Storage Model

DATA	STORAGE TYPE	RATIONALE
Admin, config, supply cap, metadata	Instance	Shared config, high read frequency, extends with contract TTL
Investor balances	Persistent	Must survive archival — investor funds cannot be lost
Allowances	Persistent	With `expiry_ledger` — Temporary storage silently deletes at TTL 0, unacceptable for securities
Allowlist / Blocklist	Persistent	Regulatory data, must never expire silently

2.3 Revenue Distribution Engine — Port from EVM

Tokeshare already distributes real USDC revenue on EVM chains via a push-based `BatchDistributor` contract (`0x8c1eCBa...` on Polygon). The Soroban version adopts a **pull-based** architecture — the operator deposits revenue, and holders claim their pro-rata share individually. This design is significantly better suited to Soroban's resource model (bounded CPU per invocation, no unbounded loops).



EVM vs Soroban — Side by Side

EVM (CURRENT — POLYGON/BASE)

```
// Push-based: operator pays gas for all
// wagmi/viem call
writeContract({
  address: '0x8c1eCBa...',
  abi: BATCH_DISTRIBUTOR_ABI,
  functionName: 'distribute',
  args: [
    '0x8335...USDC',
    recipients,  // address[]
    amounts      // uint256[]
  ]
})
// Max 200 recipients per tx
// Operator bears all gas costs
// Fails entirely if one transfer reverts
```

SOROBAN (NEW — STELLAR)

```
// Pull-based: holder claims their share
// Step 1: Operator deposits revenue
deposit_revenue(operator, asset_id, amount)

// Step 2: Each holder claims
claim(holder, dist_id)
  → calculates pro-rata share
  → transfers USDC to holder
  → marks as claimed

// No recipient limit
// Each holder pays own fees (minimal)
// Atomic per claim, no cascade failure
```

deposit_revenue()

RUST / SOROBAN

```
fn deposit_revenue(e: Env, operator: Address, asset_id: u64, amount: i128) {
    operator.require_auth();
    require_operator(&e, &operator);

    // Transfer USDC from operator to contract
    let usdc = token::TokenClient::new(&e, &load(&e, &DataKey::UsdcSac));
    usdc.transfer(&operator, &e.current_contract_address(), &amount);

    // Snapshot total supply at distribution time
    let rwa = token::TokenClient::new(&e, &load_asset_token(&e, asset_id));
    let snapshot = rwa.total_supply();

    // Record distribution
    let dist_id = increment_counter(&e);
    e.storage().persistent().set(
        &DataKey::Distribution(dist_id),
        &DistRecord {
            asset_id,
            amount,
            supply_snapshot: snapshot,
            ledger: e.ledger().sequence(),
            timestamp: e.ledger().timestamp(),
        },
    );

    DistEvents::deposited(&e, dist_id, asset_id, amount);
}
```

claim()

RUST / SOROBAN

```
fn claim(e: Env, holder: Address, dist_id: u64) -> i128 {
    holder.require_auth();

    let dist = load_distribution(&e, dist_id);

    // Prevent double-claim
    if e.storage().persistent().has(&DataKey::Claimed(dist_id, holder.clone())) {
        panic_with_error!(&e, Error::AlreadyClaimed);
    }

    // Calculate pro-rata share: (holder_balance * total_revenue) / total_supply
    let balance = token::TokenClient::new(
        &e, &load_asset_token(&e, dist.asset_id)
    ).balance(&holder);
    let share = (balance * dist.amount) / dist.supply_snapshot;

    // Transfer USDC to holder
    let usdc = token::TokenClient::new(&e, &load(&e, &DataKey::UsdcSac));
    usdc.transfer(&e.current_contract_address(), &holder, &share);

    // Mark as claimed
    e.storage().persistent().set(&DataKey::Claimed(dist_id, holder.clone()), &true);

    share
}
```

Safety Guarantees

- **Supply snapshot** — frozen at deposit time, prevents dilution attacks (minting after deposit to claim larger share)
- **Claimed flag** — persistent storage prevents double-claiming across ledger boundaries
- **Pull-based** — no unbounded loops, each claim is O(1), compatible with Soroban resource limits
- **Operator gating** — only authorized operators can deposit, prevents griefing via dust deposits

2.4 USDC Settlement

All payments settle in Circle-issued USDC via the **Stellar Asset Contract (SAC)**. This is the canonical USDC on Stellar — not a wrapped or bridged version.

PARAMETER	VALUE
Mainnet SAC Address	`CBIELTK6YBZJU5UP2WWQEUCYKLPU6AUNZ2BQ4WWFEIE3USCIHMXQDAMA`
Issuer	Circle (native Stellar asset)
Decimals	7
Integration pattern	`pay.transfer(&buyer, &treasury, &pay_cost)`
Already in codebase	Yes — existing sale contract uses SAC for all payments

3. Integration Architecture (8 Integrations)

3.1 Stellar Wallets Kit

Status: Already integrated | **Package:** `@creit.tech/stellar-wallets-kit`

```
User clicks "Connect Wallet" → openModal() → wallet selection (Lobstr / Freighter / xBull) → signTransaction() → submit to Soroban RPC → confirmation
```

Existing code: `lib/stellar.ts` (157 lines), `hooks/useTresSale.ts` (80 lines), `config/stellar.ts` (33 lines)

Extensions for production: multi-asset browsing across all listed RWAs, automatic trustline management for new RWA tokens, portfolio dashboard with claim history.

3.2 Privy

Purpose: Onboard non-crypto LATAM users via email/social login with embedded Stellar wallets

```
User enters email → Privy SDK → embedded Stellar keypair generated → custodial model → signTransaction() via Privy → submit to Soroban RPC
```

Dual-path architecture: Crypto-native users connect via Stellar Wallets Kit. Non-crypto users sign up with Privy. Both paths converge on the same Soroban contracts — the frontend detects which signer is active and routes accordingly.

3.3 MoneyGram Access

Purpose: Cash-in/cash-out for unbanked or cash-preferred LATAM investors | **Coverage:** 400K+ agent locations worldwide

```
Cash-in: Investor visits MoneyGram agent → deposits local fiat → USDC credited to Stellar wallet → invest in RWA token
Cash-out: Investor claims USDC revenue → sends USDC to MoneyGram → collects local currency at nearest agent
```

Integration: KYB registration at T0 → Developer Portal API access → programmatic cash-in/cash-out flows.

3.4 alfredpay

Purpose: Direct bank transfers in LATAM local currencies (DOP, ARS, COP)

```
Investor requests withdrawal → alfredpay API → USDC converted → DOP/ARS/COP bank transfer → investor's local bank account
```

Complementary to MoneyGram: MoneyGram handles cash; alfredpay handles bank transfers. Together they cover both banked and unbanked LATAM populations.

3.5 Stellar Disbursement Platform (SDP)

Purpose: Bulk USDC revenue distribution to all token holders at scale

```
Operator collects revenue from asset → SDP bulk payment API → USDC distributed to all holder wallets → operator dashboard for tracking & audit
```

Scale advantage: SDP is battle-tested by SDF for disbursements to thousands of recipients. Used alongside the pull-based Revenue Distribution Engine — SDP handles the off-chain orchestration and notification layer while the contract handles on-chain accounting.

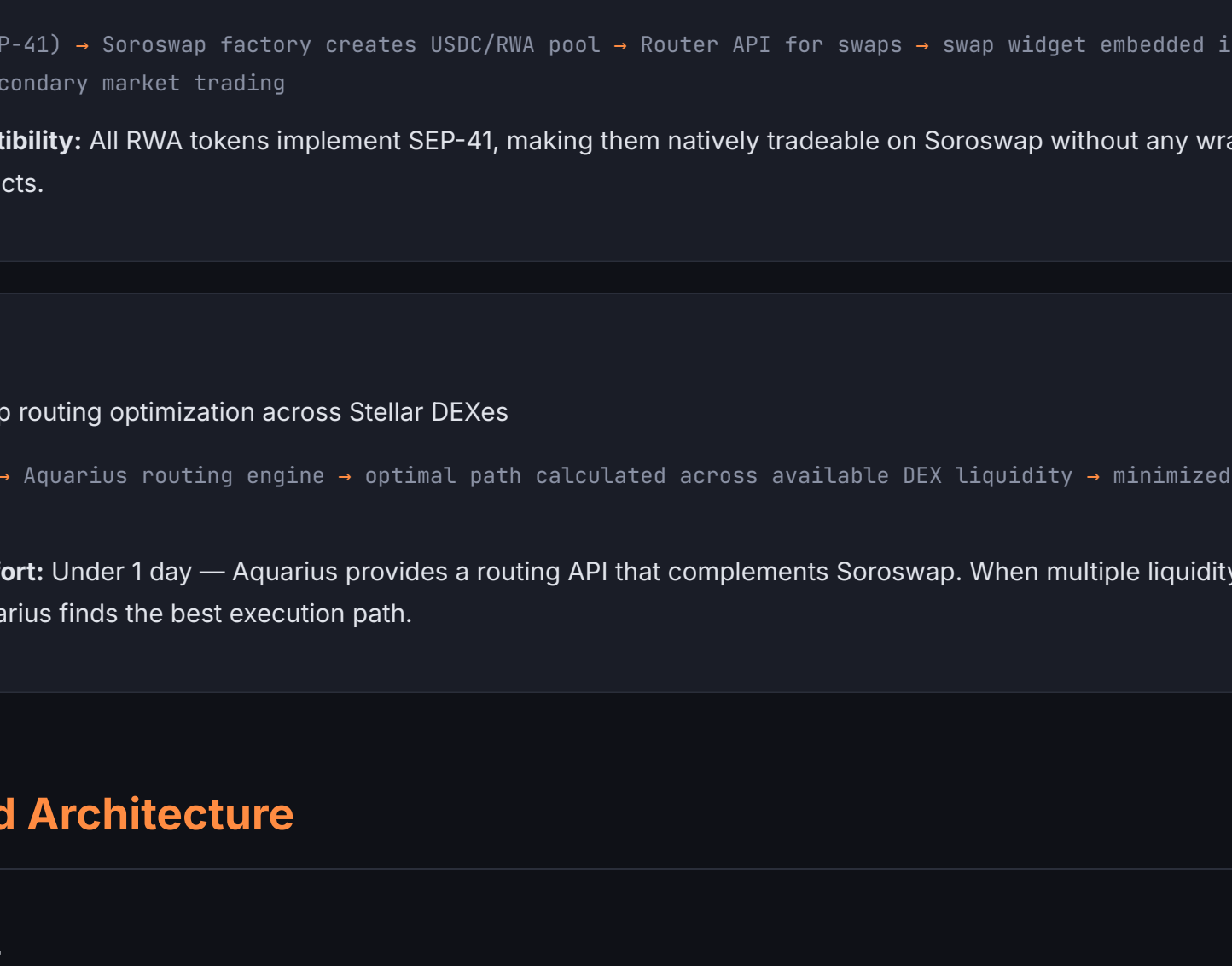
3.6 CCTP (Cross-Chain Transfer Protocol)

Purpose: Bridge native USDC from EVM chains (Ethereum, Base, Polygon) to Stellar for investment

```
Connect EVM wallet → approve USDC spend → TokenMessenger.depositForBurn() on EVM → Circle attestation service → receiveMessage() on Stellar → native USDC minted on Stellar → invest in RWA token
```

Frontend UX: Burn confirmation → attestation status polling → mint completion → automatic redirect to asset purchase flow. The user sees a single unified experience.

Strategic value: Opens Tokeshare to the entire EVM DeFi ecosystem — users with USDC on Ethereum/Base/Polygon can invest without first manually bridging.


CCTP CROSS-CHAIN FLOW — EVM TO STELLAR
EVM INVESTOR
Approve USDC spend on EVM chain
EVM INVESTOR
Call depositForBurn(amount, stellar_dest)
EVM CHAIN
USDC burned on source chain
CIRCLE
Burn event detected — attestation generated
STELLAR
receiveMessage(attestation) — native USDC minted
SALE CONTRACT
buy(asset_id, amount) — RWA Token delivered to investor

3.7 Soroswap

Purpose: USDC/RWA AMM pools for secondary market liquidity

```
RWA Token (SEP-41) → Soroswap factory creates USDC/RWA pool → Router API for swaps → swap widget embedded in Tokeshare frontend → secondary market trading
```

Native compatibility: All RWA tokens implement SEP-41, making them natively tradeable on Soroswap without any wrapper or adapter contracts.

3.8 Aquarius

Purpose: Swap routing optimization across Stellar DEXes

```
Swap request → Aquarius routing engine → optimal path calculated across available DEX liquidity → minimized slippage for investor
```

Integration effort: Under 1 day — Aquarius provides a routing API that complements Soroswap. When multiple liquidity sources exist for a pair, Aquarius finds the best execution path.

4. Backend Architecture

Event Indexer

A dedicated indexer subscribes to Soroban contract events via **Soroban RPC** and **Horizon**, processing them into a PostgreSQL database for fast frontend queries.

Indexed Events

EVENT	SOURCE CONTRACT	DATA CAPTURED
`AssetCreated`	RWA Sale	asset_id, name, type, total_supply, price, token_address
`TokenPurchased`	RWA Sale	asset_id, buyer, amount, usdc_paid, timestamp
`RevenueDeposited`	Revenue Engine	dist_id, asset_id, amount, supply_snapshot
`Claimed`	Revenue Engine	dist_id, holder, share_amount
`Transfer`	RWA Token	from, to, amount (SEP-41 standard event)
`AllowlistUpdated`	RWA Token	address, status (added/removed), operator

REST API

ENDPOINT	DESCRIPTION
`GET /assets`	List all tokenized assets with current sale status, supply remaining, price
`GET /assets/:id`	Asset detail: metadata, holder count, revenue history, sale progress
`GET /holders/:address`	Portfolio for a given address: holdings, claimable revenue, transaction history
`GET /distributions`	Revenue distribution history with claim status per holder

Stack

COMPONENT	TECHNOLOGY
Runtime	Node.js
Database	PostgreSQL
Blockchain data	Soroban RPC + Horizon
Event subscription	Soroban RPC `getEvents` with cursor-based pagination

5. Security Model (STRIDE Analysis)

THREAT CATEGORY	ATTACK VECTOR	MITIGATION
Spoofing	Unauthorized admin/operator actions	`require_auth()` on every admin and operator function. Separate admin vs operator roles. Key rotation supported.
Tampering	Manipulation of balances, supply, or distribution records	Persistent storage with invariants enforced in contract logic. Supply snapshots are immutable once written. Checked arithmetic prevents overflow.
Repudiation	Denying transactions occurred	Soroban events are append-only on the ledger. All state changes emit events. Backend indexes events for complete audit trail.
Info Disclosure	Investor PII exposure	Only public keys stored on-chain. KYC data handled off-chain by compliance provider. Privy manages identity securely — never touches the contract layer.
Denial of Service	Unbounded loops, spam claims, resource exhaustion	Pull-based claims are O(1) per invocation. Operator-only deposits prevent griefing. Soroban enforces CPU/memory limits per invocation. SDP handles bulk operations off-chain.
Elevation of Privilege	Non-admin gaining admin capabilities	`require_auth()` + role separation (admin vs operator). Admin key rotation supported. Multi-sig governance planned for mainnet.

6. Existing Codebase

Soroban (Already Built)

COMPONENT	LINES	TESTS	STATUS
`soroban/contracts/sale/src/lib.rs`	187	7 passing	Live on Stellar testnet
`scripts/stellar/deploy.sh`	198	—	Complete 7-step deploy pipeline
`lib/stellar.ts`	157	—	RPC, XDR, Horizon integration
`hooks/useTresSale.ts`	80	—	React hooks for sale interaction
`config/stellar.ts`	33	—	Testnet/mainnet config
`app/poc-stellar/page.tsx`	200+	—	Live POC with Stellar Wallets Kit

EVM (Live — Reference for Port to Soroban)

CONTRACT	CHAIN	ADDRESS	STATUS
TFT (Sale)	Polygon	`0xB48F...`	Live — $43,945+ processed
BatchDistributor	Polygon	`0x8c1eCBa...`	Live — real USDC distributions
TGG Token	Polygon + Ethereum	—	Live multi-chain
Marketplace	Polygon	`0xe0F6...`	Live
TMC Token	Base	—	Live
TSP500 Token	Base	—	Live

7. Deployment Stack

COMPONENT	TECHNOLOGY
Smart contracts	Rust / Soroban SDK 26 / WASM
Frontend	Next.js 15 / TypeScript / Stellar Wallets Kit / Privy SDK / CCTP SDK / Soroswap Widget
Backend	Node.js / PostgreSQL / Soroban RPC + Horizon
Hosting	Hetzner (EU) + Vercel (frontend)
CI/CD	GitHub Actions
Repository	github.com/bim-finance-org/tokeshare

Tokeshare — LATAM Real-World Investing on Stellar

github.com/bim-finance-org/tokeshare | tokeshare.co